AMD INTERNSHIP HIRING (INTERVIEW EXPERIENCES)

Reference Links :

1. https://www.reddit.com/r/csMajors/comments/1jcmkg5/amd_interncoop_interview/
2. https://www.geeksforgeeks.org/interview-experiences/amd-interview-experience-for-internship-on-campus/
3. https://www.vervecopilot.com/interview-questions/what-unlocks-success-in-amd-internships-and-how-to-master-your-interview-performance
4. https://www.jointaro.com/interviews/companies/amd/experiences/software-engineerinternship-hyderabad-march-1-2021-no-offer-neutral-89f8b52e/
5. https://gauthamshekar.medium.com/my-internship-interview-experience-at-amd-e583ebe31728
6. https://www.glassdoor.co.in/Interview/AMD-Internship-Interview-Questions-EI_IE15.0,3_KO4,14.htm
7. https://www.geeksforgeeks.org/gfg-academy/amd-recruitment-process/
8. https://akash-kumar916.medium.com/amd-internship-interview-experience-15ae4ec7569c
9. https://www.geeksforgeeks.org/interview-experiences/amd-internship-interview-experience-on-campus-2021/

Based on the detailed interview experiences from the links provided (Reddit, GeeksforGeeks, Medium, Verve Copilot, etc.), here is the comprehensive curation of **Do's and Don'ts** and the **complete list of every single interview question** reported by candidates.

Part 1: The Do's and Don'ts

Do's:

- **Know Your Resume Inside Out:** You will be grilled on every project, technology, and line on your resume. Be ready to explain *why* you made specific design choices (e.g., "Why did you use a specific activation function in your CNN?").
- **Master the Fundamentals:** For AMD, "basics" means deep knowledge of C/C++ memory management (pointers, stack vs heap) and Computer Architecture (caches, pipelines), not just LeetCode.
- **Prepare for "Hybrid" Roles:** Even for software roles, expect hardware-adjacent questions (bits, bytes, memory layout). Even for hardware roles, expect scripting/coding questions (Python/C).
- **Use the STAR Method:** For behavioral questions, structure answers as **S**ituation, **T**ask, **A**ction, **R**esult.
- **Be Honest:** If you don't know an answer, admit it but try to reason through it aloud. Interviewers value your thought process over the correct final answer.
- **Show Passion for Hardware:** Mention interest in semiconductors, chip design, or low-level optimization.

Don'ts:

- **Don't fake knowledge:** If you claim to know Verilog or C++ on your resume, they will test you on the nuances (e.g., volatile, virtual functions).
- **Don't ignore the "Why":** deeply understanding *why* a piece of code works (e.g., how malloc actually finds free memory) is more important than just writing it.
- **Don't rush coding:** In whiteboard/virtual coding, clarify requirements (edge cases, input size) before you start writing.
- **Don't neglect soft skills:** Communication is key. Silent problem solving is a red flag. Talk through your logic.

Part 2: The Complete Question Bank

I have combined *every single question* found across all your sources. They are categorized for clarity, but nothing has been removed.

I. Behavioral & Managerial Questions

1. Tell me about yourself.
2. Explain the project you worked on. (Expect deep dives: "What was your specific contribution?", "What challenges did you face?")
3. What interests you about this position and AMD?
4. What are your greatest professional strengths and weaknesses?
5. Describe a challenging situation at work/college and how you handled it.
6. Where do you see yourself in the next 5 years?
7. Why is technology helpful for the younger generation vs. older generation? (Group Discussion Topic)
8. Are you willing to relocate?
9. Do you have any questions for us?
10. What are your hobbies?
11. Is this your first interview? Have you received other offers?
12. Why do you want to work in the semiconductor industry specifically?

II. C / C++ & Language Concepts

This is the most critical technical section for AMD software interviews.

Pointers & Memory:

13. What is a pointer? Where is it stored in memory?

14. What is the difference between a pointer and a reference?

15. What is a void pointer?

16. What is a "smart pointer" in C++? (Explain unique_ptr, shared_ptr, weak_ptr).

17. Difference between malloc() and new.

18. Code Spotting: Given a snippet ptr = malloc(n * sizeof(int));, what is the error? (Check for return value check, type casting in C++, etc.)

19. How do you free memory allocated by new? (Answer: delete vs delete[]).

20. Does a pointer allocate a virtual or physical address?

21. Without using sizeof, how do you find the size of a particular data type?

22. How do you check the size of a data type in C?

23. What is a dangling pointer?

24. What is a memory leak?

25. What is the difference between static and global variables?

26. Using static, write a C program to find the radius of a circle (maintaining state).

27. List the storage classes in C (auto, register, static, extern).

28. What is the volatile keyword? (Crucial for embedded/hardware interaction).

29. What is the const keyword? Difference between const and constexpr.

OOP & Advanced C++:

30. What is a virtual function? What is a pure virtual function?

31. What is runtime polymorphism?

32. Can you call a virtual function from a constructor?

33. Difference between an Abstract Class and an Interface (Java/C++ context).

34. What is the "Diamond Problem" in multiple inheritance?

35. What is a copy constructor? Shallow copy vs. Deep copy.

36. What is the order of constructor/destructor calls in inheritance?

III. Operating Systems & Computer Architecture

37. What is a thread? What is a process?
38. What is the difference between a thread and a process?
39. How do you ensure threads behave as expected regarding shared resources? (Synchronization, Mutex, Semaphores).
40. What is Virtual Memory? How is it managed?
41. What is address translation? (Logical to Physical).
42. What happens when the system boots?
43. What is a boot loader? What does it do?

44. What is Cache? Why do we need it?
45. Explain Cache Coherency.
46. What is a deadlock?
47. Linux Commands:
    - Command to check RAM size.
    - Command to list files with their sizes.
    - Command to check CPU usage.
48. What is SDLC (Software Development Life Cycle)?

IV. Coding & Data Structures (DSA)

49. **Linked List:** Insert a node at the middle of a linked list efficiently.
50. **Array:** Find the max element in an array in O(1) time. (Trick question: discussion involved sqrt decomposition or parallel processing concepts).
51. **Recursion:** Write recursive code for the factorial of a number.
52. **Loops:** Print 1 to 100 *without* using any loop. (Use recursion or goto).
53. **List Comprehension (Python):** Given [2,5,3,4,6,9,10], print numbers at even indices using list comprehension.
54. **Dictionary (Python):** Given list [1,2,3,4,5], output squares [1,4,9,16,25] using a dictionary.
55. **String:** Given string "MALAYALAM", output "MALY" (remove duplicates).
56. **String:** Remove duplicates *without* using a dictionary/hash map.
57. **Bit Manipulation:** Programs related to setting/clearing bits (common in hardware roles).

V. Digital Design & Hardware (For Hybrid/ECE Roles)

58. **Verilog:** What is the difference between blocking (=) and non-blocking (<=) assignments?
59. **Verilog:** Difference between task and function.
60. **Verilog:** Difference between $display and $monitor.
61. **Digital Logic:** Design a logic circuit to multiply two 4-bit numbers.
62. **Digital Logic:** Implement a circuit to check if a register has a non-zero value.
63. **Digital Logic:** How do PUSH and POP instructions work in a stack?
64. **Concepts:** What is a Flip-Flop? Difference between Latch and Flip-Flop.
65. **Concepts:** Explain Setup time and Hold time.
66. **Concepts:** What is Clock Domain Crossing (CDC)?
67. **Concepts:** What is a Ring Oscillator?
68. **Verification:** How do you verify a design? What is a testbench?

69. **Verification:** Difference between Code Coverage and Functional Coverage.

VI. Python / AI / ML (Specific to Resume Projects)

70. **CNNs:** What layers did you use in your Convolutional Neural Network?
71. **CNNs:** What is a Max Pooling layer?
72. **CNNs:** Where does the Fully Connected layer come (after/before pooling)?
73. **Activation Functions:** List them. Difference between Sigmoid and ReLU.
74. **Python:** What are Generators in Python?
75. **Python:** Functional programming advantages.

VII. Java (If on Resume)

76. What is Garbage Collection? How does it work?
77. Explain Generational Garbage Collection.
78. What is the difference between an Interface and an Abstract Class?